RUPEEWISE

Monthly Expense Tracker

DBMS Experiential Learning — Level 3 Project Report

Submitted by:
Nakshathira Bharathi
SRN: PES2UG24AM096

Course: Database Management Systems

Department: AIML 5B

ABSTRACT

Managing personal finances effectively requires individuals to track their income, expenses, budgets, and savings goals. Manual tracking can make it difficult to understand spending patterns, monitor monthly limits, and maintain organized financial records. To address these challenges, **RupeeWise — Monthly Expense Tracker** is developed as a web-based financial management application.

The project uses **Next.js and TypeScript** to provide the application interface and backend API functionality. **MySQL** is used to store structured relational data, including user accounts, transaction categories, financial transactions, monthly budgets, category-specific budgets, and savings goals. **MongoDB** is integrated to store bank statement import batches in a flexible document format.

The system incorporates user authentication, transaction management, monthly financial summaries, budgeting, savings goal tracking, and statement import functionality. Advanced SQL features, including a view, a stored procedure, and insert and update triggers, are implemented to support financial reporting and maintain data consistency. Database constraints and application-level validation further help preserve data integrity and protect user-specific records.

The project demonstrates the integration of relational and NoSQL databases within a functional web application. It connects database design concepts, SQL operations, advanced database features, API integration, and security practices to address a practical financial management problem.

Keywords: Personal Finance Management, Expense Tracking, MySQL, MongoDB, Next.js, Stored Procedures, Database Triggers, Web Application.

1. INTRODUCTION

1.1 Background

Personal financial management is an important part of maintaining financial stability. Individuals regularly receive income and spend money on various activities such as food, transportation, education, shopping, and other necessities. Without a proper tracking system, it can become difficult to understand spending habits, compare income with expenses, and manage monthly budgets effectively.

Traditional methods, such as handwritten records and basic spreadsheets, require users to maintain information manually. As financial records increase, organizing transactions and generating meaningful summaries can become time-consuming. A digital expense tracking system can simplify these tasks by maintaining structured records and providing convenient access to financial information.

1.2 Introduction to RupeeWise

RupeeWise is a web-based monthly expense tracking application developed to help users organize and monitor their personal finances. The system provides an interface for managing income and expense transactions, reviewing financial summaries, setting monthly budgets, and working with imported bank statement data.

The application uses Next.js and TypeScript for its web interface and backend API functionality. MySQL serves as the primary relational database for structured financial information, while MongoDB stores imported statement batches in a flexible document format. This combination demonstrates the practical integration of SQL and NoSQL database technologies.

1.3 Importance of the Project

RupeeWise demonstrates how database management concepts can be applied to a real-world financial problem. Instead of relying only on basic data entry and retrieval, the project incorporates database relationships, integrity constraints, indexes, aggregate queries, a view, a stored procedure, and validation triggers.

The application also demonstrates database–application integration through API routes, environment-based configuration, and authenticated access to user-specific records. The use of two database paradigms allows structured financial information and flexible statement-import data to be managed according to their different requirements.

Overall, RupeeWise connects database design, implementation, and web application development to provide a practical foundation for personal financial tracking and reporting.

2. PROBLEM STATEMENT, OBJECTIVES AND SCOPE

2.1 Problem Statement

Managing personal finances involves keeping track of income, expenses, monthly spending limits, and financial goals. When this information is recorded manually or maintained across multiple files, users may find it difficult to organize transactions, calculate monthly totals, identify spending patterns, and monitor their budgets consistently.

There is a need for a centralized application that allows users to maintain financial records, obtain monthly summaries, manage budgets, and process bank statement data. The system must also maintain data consistency, protect user-specific financial information, and support reliable storage and retrieval of records.

RupeeWise addresses this problem through a web-based financial tracking application that integrates relational and NoSQL databases with a functional application interface.

2.2 Objectives

The primary objectives of RupeeWise are:

1. To develop a web-based application for recording and managing personal income and expenses.
2. To provide user registration, login, session management, and logout functionality.
3. To organize transactions using categories and transaction types.
4. To generate financial summaries that help users understand monthly income and expenditure.

5. To support monthly budget management and provide a foundation for monitoring spending.
6. To store and manage bank statement import batches for review and processing.
7. To use MySQL for structured financial records and MongoDB for flexible statement-import documents.
8. To implement advanced SQL features, including views, stored procedures, and triggers.
9. To maintain data integrity through database constraints, input validation, and appropriate relationships.
10. To demonstrate the integration of database concepts with a modern web application.

2.3 Scope of the Project

The project covers the design, implementation, and integration of a personal expense tracking application.

The system includes user authentication, transaction management, financial summaries, monthly budgeting, and statement import functionality. It uses MySQL to maintain users, categories, transactions, monthly budgets, category budgets, and savings-goal records. MongoDB stores bank statement import batches and their associated rows.

The database implementation includes primary keys, foreign keys, unique constraints, check constraints, indexes, a monthly financial summary view, a stored procedure, and insert and update triggers for transaction category validation.

The application communicates with the databases through backend API routes. Environment variables are used for database configuration, and authenticated requests are used to restrict access to user-specific information.

The scope is limited to personal financial record management and basic reporting. Features such as direct bank-account integration, automatic bank synchronization, investment management, and AI-powered financial advice are outside the current implementation and may be considered as future enhancements.

3. FUNCTIONAL AND NON-FUNCTIONAL REQUIREMENTS

3.1 Functional Requirements

The functional requirements describe the operations and services provided by RupeeWise.

FR1: User Authentication

The system shall allow users to register, log in, maintain an authenticated session, and log out. Passwords shall be stored as hashes rather than plain-text values.

FR2: Transaction Management

The system shall allow authenticated users to create, view, update, and delete their income and expense transactions. Each transaction shall contain relevant details such as category, amount, date, description, and optional notes.

FR3: Transaction Categorization

The system shall associate transactions with predefined categories. It shall validate that the transaction type matches the selected category type.

FR4: Financial Summaries

The system shall calculate financial summaries, including income, expenses, and net balance, for a selected user and month.

FR5: Monthly Budget Management

The system shall allow users to set and update a monthly budget. The database shall prevent duplicate monthly budget entries for the same user and month.

FR6: Savings Goal Storage

The system shall maintain savings goals associated with users, including target amounts, current savings, optional target dates, and goal statuses.

FR7: Statement Import

The system shall allow authenticated users to submit bank statement data as JSON rows. The import API shall validate the request and store submitted batches in MongoDB for review and processing.

FR8: Import History

The system shall allow authenticated users to retrieve their recent statement import batches, including status, import date, row count, and a limited preview of imported rows.

FR9: Database Integration

The system shall communicate with MySQL and MongoDB through backend API routes and dedicated database connection modules.

FR10: Data Integrity

The system shall use database constraints and triggers to prevent invalid monetary amounts, invalid references, duplicate records where prohibited, and inconsistent transaction category types.

3.2 Non-Functional Requirements

NFR1: Security

Sensitive database configuration shall be stored in environment variables. Passwords shall be hashed, and protected API endpoints shall require an authenticated session.

NFR2: Data Integrity

Primary keys, foreign keys, unique constraints, check constraints, and triggers shall help maintain the consistency of stored records.

NFR3: Reliability

The application shall handle invalid requests and database errors by returning appropriate error responses rather than exposing internal error details to the user.

NFR4: Maintainability

The project shall separate application pages, API routes, and database connection logic into organized modules to make the code easier to understand and maintain.

NFR5: Usability

The application shall provide a web interface through which users can manage financial records and access relevant financial information.

NFR6: Configuration

Database connection settings shall be configured through environment variables rather than hard-coded credentials.

NFR7: Data Isolation

Authenticated users shall access only the financial records and statement import batches associated with their own accounts through protected operations.

NFR8: Extensibility

The database structure and modular application design shall provide a foundation for future features, such as statement-file uploads, additional financial reports, and more detailed spending analysis.

3.3 Summary

The functional requirements define the core financial operations supported by RupeeWise, while the non-functional requirements address security, integrity, reliability, usability, and maintainability. Together, these requirements guide the design and implementation of the application and its database components.

4. TECHNOLOGY STACK AND SYSTEM ARCHITECTURE

4.1 Technology Stack

RupeeWise uses a modern web development stack that combines a web interface, backend API functionality, and two database technologies.

Technology	Purpose
Next.js	Framework for the web application and backend API routes.
React	Component-based user interface development.
TypeScript	Type-safe application development.
Tailwind CSS	Styling and layout of the user interface.
MySQL	Relational storage for users, transactions, categories, budgets, and savings goals.
MongoDB	Document storage for bank statement import batches.
Node.js	Runtime environment for the application and server-side functionality.
MongoDB Node.js Driver	Establishes connections to MongoDB and performs document operations.
Git and GitHub	Version control, source code management, and project hosting.

4.2 System Architecture

RupeeWise follows a web application architecture in which the user interface communicates with backend API routes, and the backend interacts with the appropriate database.

The architecture consists of the following layers:

1. Presentation Layer

The presentation layer consists of the Next.js application pages and React components. It provides interfaces for authentication, transaction management, budgeting, financial summaries, and statement imports.

2. Application and API Layer

Next.js route handlers process requests from the interface. They validate incoming data, check authentication where required, apply application logic, and communicate with the databases through dedicated connection modules.

3. Authentication and Security Layer

The authentication module manages session verification and protected access. The application uses signed session tokens stored in an HTTP-only cookie. Protected operations use the authenticated user's identity to restrict access to user-specific data.

4. Relational Database Layer

MySQL stores the application's structured financial records. Its tables, keys, constraints, indexes, views, stored procedure, and triggers support reliable storage, retrieval, reporting, and validation.

5. NoSQL Database Layer

MongoDB stores bank statement import batches in the `bank_statement_imports` collection. Each document contains information about the import batch and its submitted statement rows.

4.3 Data Flow

The application follows these general data flows:

MySQL transaction workflow:
User Interface → Next.js API Route → Authentication and Validation → MySQL → API Response → User Interface

MongoDB statement import workflow:
User Interface → Import API Route → Session and Input Validation → MongoDB → Import Result → User Interface

Statement conversion workflow:
Imported statement rows are reviewed and processed for conversion into structured MySQL transaction records.

These flows separate user interaction from database operations and allow each database to handle the type of data for which it is used.

4.4 Database Technology Justification

MySQL is suitable for financial records because it supports structured tables, primary and foreign keys, constraints, joins, aggregate queries, and advanced SQL features. These capabilities help maintain relationships between users, transactions, categories, budgets, and savings goals.

MongoDB is suitable for statement import batches because its document model can store collections of imported rows whose fields may vary between statements. It allows the original submitted batch to

be retained separately from the structured financial transactions maintained in MySQL.

Using both databases demonstrates how relational and document-oriented storage can work together within a single application.

4.5 Summary

The architecture of RupeeWise separates the user interface, API logic, authentication, and data storage. Next.js connects the application to MySQL and MongoDB, while the two databases serve distinct purposes: MySQL manages structured financial data, and MongoDB stores statement-import documents. This design demonstrates practical database–application integration beyond basic CRUD operations.

5. DATABASE DESIGN AND ER DIAGRAM

5.1 Overview of Database Design

The RupeeWise database is designed to organize financial information while maintaining relationships and data integrity. The primary relational database is MySQL, named `monthly_expense_tracker`.

The database consists of six tables: `users`, `categories`, `transactions`, `monthly_budgets`, `category_budgets`, and `savings_goals`. These tables represent the main entities required for managing user accounts, income and expenses, budgets, and savings targets.

Each table has a primary key to identify its records. Foreign keys connect related tables, while unique constraints, check constraints, and indexes support data integrity and efficient retrieval.

5.2 Entity Relationship Diagram

The Entity Relationship (ER) diagram illustrates the entities, their attributes, primary keys, foreign keys, and relationships in the RupeeWise database.

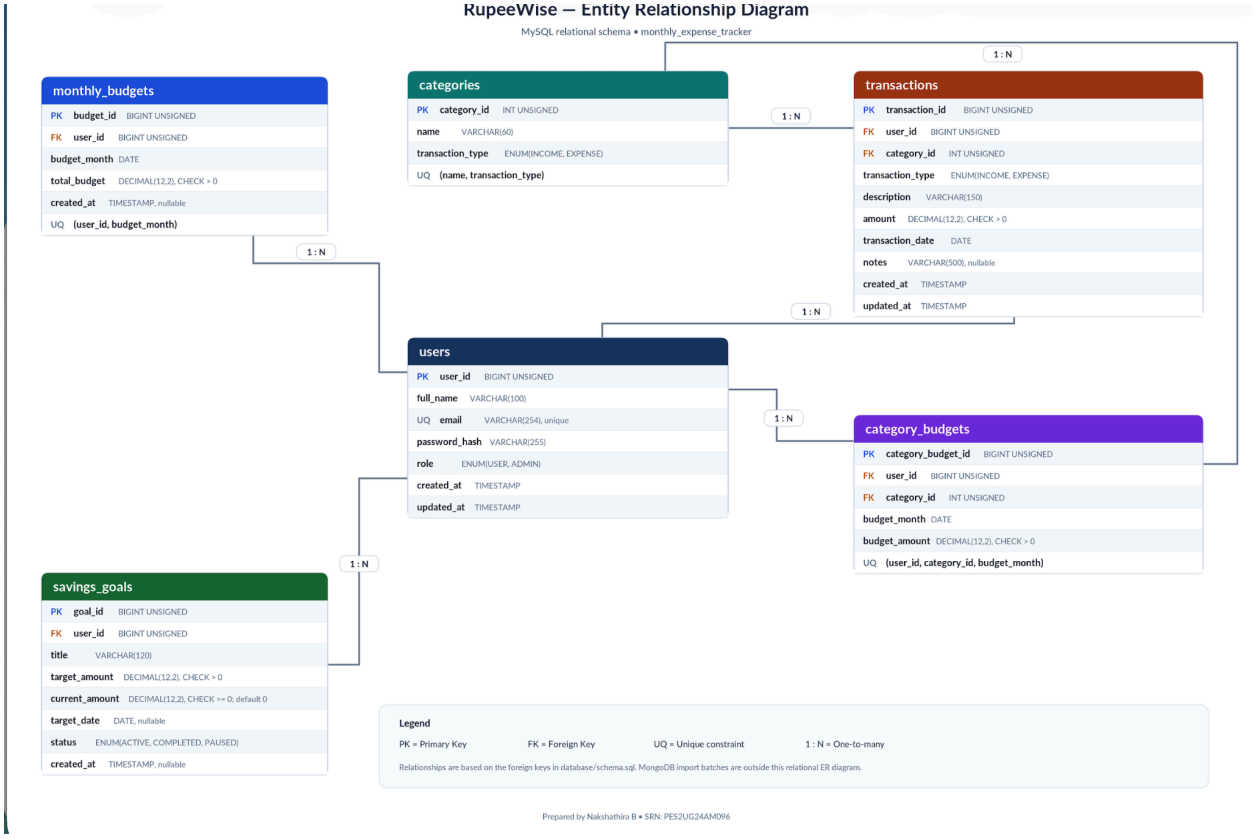


Figure 1: Entity Relationship Diagram of RupeeWise

5.3 Entities and Their Purpose

1. Users

The `users` table stores account information, including the user's name, email, password hash, role, and timestamps. Each user is uniquely identified by `user_id`.

2. Categories

The `categories` table defines the available income and expense categories. The `transaction_type` attribute identifies whether a category is intended for income or expenses.

3. Transactions

The `transactions` table stores financial records, including the user, category, transaction type, description, amount, date, and optional notes. Each transaction references a user and a category.

4. Monthly Budgets

The `monthly_budgets` table stores each user's total budget for a particular month. A unique constraint prevents multiple monthly budget records for the same user and month.

5. Category Budgets

The `category_budgets` table stores budget allocations for specific categories and months. It links users with categories and prevents duplicate allocations for the same user, category, and month.

6. Savings Goals

The `savings_goals` table stores users' savings targets, current saved amounts, optional target dates, and goal statuses.

5.4 Relationships Between Entities

The database contains the following one-to-many (1) relationships:

1. **Users to Transactions:** One user can have multiple transactions, but each transaction belongs to one user.
2. **Categories to Transactions:** One category can be used by multiple transactions, but each transaction references one category.
3. **Users to Monthly Budgets:** One user can have monthly budget records for multiple months.
4. **Users to Category Budgets:** One user can have multiple category-budget records.

5. **Categories to Category Budgets:** One category can be referenced by multiple category-budget records.
6. **Users to Savings Goals:** One user can maintain multiple savings goals.

These relationships are enforced through foreign key constraints in MySQL.

5.5 Keys and Constraints

The database implements the following integrity mechanisms:

- **Primary keys:** Uniquely identify records in all six tables.
- **Foreign keys:** Connect transactions and budgets to users and categories, and savings goals to users.
- **Unique constraints:** Prevent duplicate email addresses, duplicate category name/type combinations, duplicate user-month budgets, and duplicate user-category-month budgets.
- **Check constraints:** Require positive transaction amounts and budget amounts, positive savings targets, and non-negative current savings amounts.
- **ENUM constraints:** Restrict user roles, transaction types, and savings goal statuses to specified values.
- **Indexes:** Support selected lookups and transaction queries.

These constraints reduce invalid records and help preserve consistency between related financial records.

5.6 Summary

The RupeeWise ER design represents the application's main financial entities and their relationships. The relational schema translates this design into six MySQL tables connected through primary and foreign

keys. Together, the ER diagram and implemented constraints provide the foundation for consistent data storage, transaction management, financial reporting, and budgeting.

6. RELATIONAL SCHEMA AND DATA DICTIONARY

6.1 Relational Schema

The relational schema of RupeeWise is derived from its Entity Relationship diagram. It defines the six MySQL tables, their attributes, primary keys, foreign keys, and integrity constraints.

The following notation is used:

- **PK:** Primary Key
- **FK:** Foreign Key
- **UNIQUE:** Prevents duplicate values or combinations
- **NOT NULL:** Requires a value
- **CHECK:** Restricts values according to a condition

1. USERS

`USERS(user_id, full_name, email, password_hash, role, created_at, updated_at)`

- **Primary Key:** `user_id`
- **Unique Attribute:** `email`
- **Role:** ENUM('USER', 'ADMIN')
- **Purpose:** Stores account details, password hashes, user roles, and timestamps.

2. CATEGORIES

`CATEGORIES(category_id, name, transaction_type)`

- **Primary Key:** `category_id`
- **Unique Constraint:** `(name, transaction_type)`

- **Transaction Type:** ENUM('INCOME', 'EXPENSE')
- **Purpose:** Stores the categories used to classify income and expenses.

3. TRANSACTIONS

`TRANSACTIONS(transaction_id, user_id, category_id, transaction_type, description, amount, transaction_date, notes, created_at, updated_at)`

- **Primary Key:** `transaction_id`
- **Foreign Keys:** `user_id` → `USERS(user_id)`; `category_id` → `CATEGORIES(category_id)`
- **Check Constraint:** `amount > 0`
- **Indexes:** `(user_id, transaction_date)` and `category_id`
- **Purpose:** Stores individual income and expense records.

4. MONTHLY_BUDGETS

`MONTHLY_BUDGETS(budget_id, user_id, budget_month, total_budget, created_at)`

- **Primary Key:** `budget_id`
- **Foreign Key:** `user_id` → `USERS(user_id)`
- **Unique Constraint:** `(user_id, budget_month)`
- **Check Constraint:** `total_budget > 0`
- **Purpose:** Stores each user's total budget for a particular month.

5. CATEGORY_BUDGETS

`CATEGORY_BUDGETS(category_budget_id, user_id, category_id, budget_month, budget_amount)`

- **Primary Key:** `category_budget_id`
- **Foreign Keys:** `user_id` → `USERS(user_id)`; `category_id` → `CATEGORIES(category_id)`
- **Unique Constraint:** `(user_id, category_id, budget_month)`
- **Check Constraint:** `budget_amount > 0`
- **Purpose:** Stores budget allocations for individual categories and months.

6. SAVINGS_GOALS

`SAVINGS_GOALS(goal_id, user_id, title, target_amount, current_amount, target_date, status, created_at)`

- **Primary Key:** `goal_id`
- **Foreign Key:** `user_id` → `USERS(user_id)`
- **Check Constraints:** `target_amount > 0`; `current_amount >= 0`
- **Default Values:** `current_amount = 0.00`; `status = 'ACTIVE'`
- **Status:** ENUM('ACTIVE', 'COMPLETED', 'PAUSED')
- **Purpose:** Stores savings targets and tracks progress toward financial goals.

6.2 Data Dictionary

A data dictionary describes the attributes of each table, including their data types, constraints, and purposes.

The primary data types used in RupeeWise are:

- **BIGINT UNSIGNED:** Stores non-negative integer identifiers with a large range.
- **INT UNSIGNED:** Stores non-negative integer values, including category identifiers.
- **VARCHAR:** Stores variable-length text, such as names, emails, descriptions, and notes.
- **DECIMAL(12,2):** Stores monetary amounts with two decimal places.
- **DATE:** Stores transaction dates, budget months, and optional savings target dates.
- **TIMESTAMP:** Records creation or update times where defined.
- **ENUM:** Restricts a column to a predefined set of values.

The `users` table uses a unique email address to prevent duplicate accounts. The `transactions` table links users and categories through foreign keys and requires positive transaction amounts. The budget tables prevent duplicate allocations for the specified user and period. The `savings_goals` table enforces positive targets and non-negative saved amounts.

The complete column-level data dictionary, including every column's exact data type, nullability, default value, keys, indexes, and constraints, is maintained in the supplementary file `documentation/Data_Dictionary.md`.

6.3 Integrity Constraints

The relational schema maintains data consistency through the following mechanisms:

1. **Entity integrity:** Primary keys uniquely identify records.
2. **Referential integrity:** Foreign keys ensure that related records reference existing parent records.

3. **Uniqueness:** Unique constraints prevent duplicate values or combinations where required.
4. **Domain integrity:** ENUM and CHECK constraints restrict stored values to valid domains.
5. **Default values:** Defaults are provided for fields such as user role, savings progress, goal status, and timestamps where defined.

These mechanisms help ensure that financial records are stored consistently and that relationships between the database entities remain valid.

7. SQL IMPLEMENTATION AND IMPORTANT QUERIES

7.1 Overview of SQL Implementation

MySQL is the primary relational database used by RupeeWise. The database is named `monthly_expense_tracker` and contains six tables: `users`, `categories`, `transactions`, `monthly_budgets`, `category_budgets`, and `savings_goals`.

The SQL implementation uses Data Definition Language (DDL) to create and manage database structures and Data Manipulation Language (DML) to insert, retrieve, update, and delete records. It also incorporates advanced SQL functionality for reporting and validation.

7.2 Data Definition Language (DDL)

DDL statements define the database structure and its integrity rules.

The project uses the following SQL operations:

- `CREATE DATABASE` establishes the database.
- `CREATE TABLE` defines tables, columns, data types, and constraints.
- `ALTER TABLE` can be used to modify an existing table structure when necessary.
- `DROP` removes database objects when scripts need to recreate them.
- Primary keys and foreign keys establish record identity and relationships.
- `UNIQUE` and `CHECK` constraints enforce additional data integrity rules.

- Indexes support selected data retrieval operations.

The implemented table definitions are maintained in `database/schema.sql`.

7.3 Data Manipulation Language (DML)

DML supports the application's financial record management operations.

7.3.1 SELECT

The SELECT statement retrieves transaction records.

```
SELECT
    transaction_id,
    transaction_type,
    description,
    amount,
    transaction_date
FROM transactions
WHERE user_id = 1
ORDER BY transaction_date DESC;
```

This query retrieves a user's transactions and displays the most recent dates first. The user ID is illustrative and should be replaced with an existing ID when testing.

7.3.2 INSERT

The INSERT statement adds a new record to a database table.

```
INSERT INTO transactions
    (user_id, category_id, transaction_type,
     description, amount, transaction_date)
VALUES
```

```
    (1, 1, 'EXPENSE',
     'Example expense', 250.00, '2026-10-01');
```

This example demonstrates the insertion of an expense transaction. Before execution, use an existing user ID and a category ID whose type matches the specified transaction type. The insert trigger validates category/type consistency.

7.3.3 UPDATE

The UPDATE statement modifies an existing record.

```
UPDATE transactions
SET amount = 300.00
WHERE transaction_id = 1
  AND user_id = 1;
```

This example changes the amount for a specified transaction owned by the selected user. The identifiers are illustrative; the record should be verified before running the query.

7.3.4 DELETE

The DELETE statement removes an existing record.

```
DELETE FROM transactions
WHERE transaction_id = 1
  AND user_id = 1;
```

This example demonstrates deleting a transaction belonging to a specified user. The application uses authenticated ownership checks for its protected transaction operations. The SQL example should only be executed against an intended test record.

7.4 JOIN and Aggregate Queries

JOIN and aggregate queries allow the application to combine related records and derive useful financial information.

7.4.1 Transactions with Category Names

```
SELECT
    t.transaction_id,
    t.description,
    t.transaction_type,
    c.name AS category_name,
    t.amount,
    t.transaction_date
FROM transactions t
INNER JOIN categories c
    ON t.category_id = c.category_id
WHERE t.user_id = 1
ORDER BY t.transaction_date DESC;
```

This query joins the `transactions` and `categories` tables to display each transaction alongside its category name.

7.4.2 Monthly Income and Expense Totals

```
SELECT
    user_id,
    DATE_FORMAT(transaction_date, '%Y-%m') AS
transaction_month,
    SUM(
        CASE
            WHEN transaction_type = 'INCOME'
            THEN amount ELSE 0
        END
    ) AS total_income,
    SUM(
        CASE
```

```
            WHEN transaction_type = 'EXPENSE'
            THEN amount ELSE 0
        END
    ) AS total_expenses,
    COUNT(*) AS transaction_count
FROM transactions
GROUP BY
    user_id,
    DATE_FORMAT(transaction_date, '%Y-%m')
ORDER BY transaction_month DESC;
```

This query uses `GROUP BY`, `SUM()`, `COUNT()`, and `CASE` to calculate monthly income, expenses, and transaction counts for each user.

7.4.3 Users with Transactions Above a Threshold

```
SELECT
    user_id,
    COUNT(*) AS transaction_count,
    SUM(amount) AS total_transaction_amount
FROM transactions
GROUP BY user_id
HAVING SUM(amount) > 10000
ORDER BY total_transaction_amount DESC;
```

This example demonstrates `HAVING` by filtering groups whose combined transaction amounts exceed a specified threshold. It illustrates SQL aggregation and is not intended to calculate net income because it sums both income and expenses.

7.5 Advanced Feature 1: View

Name: `vw_monthly_financial_summary`

The view provides a reusable summary of monthly income, expenses, and transaction counts. It combines conditional aggregation with grouping by user and calendar month.

Example:

```
SELECT *
FROM vw_monthly_financial_summary
ORDER BY transaction_month DESC;
```

The view simplifies repeated reporting queries. It returns user-month combinations with recorded transactions.

7.6 Advanced Feature 2: Stored Procedure

Name: `sp_monthly_summary`

The stored procedure accepts a user ID and a date within the requested month. It calculates the month's income, expenses, net balance, and transaction count.

Example:

```
CALL sp_monthly_summary(1, '2026-10-01');
```

The procedure determines the first day of the selected month and the first day of the following month. It uses a `LEFT JOIN`to preserve the selected existing user even if the user has no transactions during that period.

The procedure also uses `COALESCE()` so that monetary totals are returned as zero when no matching transactions exist.

7.7 Advanced Features 3 and 4: Triggers

RupeeWise implements two validation triggers:

- `trg_transaction_category_insert`
- `trg_transaction_category_update`

Both triggers check whether the transaction type matches the type associated with the selected category. The insert trigger validates new records, while the update trigger validates changes to existing records.

If the values do not match, the trigger raises an error using `SIGNAL SQLSTATE '45000'` and the message:

```
Transaction type must match category type
```

These triggers enforce consistency at the database level instead of relying exclusively on the application interface.

7.8 Indexing and Integrity

The schema defines indexes to support selected queries, including:

- A composite index on `transactions(user_id, transaction_date)`.
- An index on `transactions(category_id)`.
- An index on `category_budgets(category_id)`.
- An index on `savings_goals(user_id)`.

Primary keys, foreign keys, unique constraints, check constraints, and ENUM columns provide further data integrity protections.

For example, transaction amounts must be greater than zero, monthly budget amounts must be positive, and savings goals must have a positive target amount and a non-negative current amount.

7.9 Summary

The SQL implementation demonstrates database structure definition, data manipulation, joins, filtering, grouping, aggregation, indexing, views, stored procedures, and triggers.

These features support financial reporting and help maintain consistent relationships between transactions, categories, users, budgets, and savings goals. The complete schema and advanced SQL definitions are available in `database/schema.sql`and `database/advanced_features.sql`, respectively.

8. MONGODB IMPLEMENTATION AND NOSQL INTEGRATION

8.1 Overview

RupeeWise integrates MongoDB with MySQL to manage different types of application data. MySQL stores structured financial records, while MongoDB stores bank statement import batches and their associated rows.

MongoDB is a document-oriented NoSQL database. Its flexible document model is suitable for imported statement data because different statements may contain different fields. The integration allows statement data to be stored and reviewed separately before it is converted into structured transaction records.

8.2 MongoDB Database and Collection

The MongoDB database used by the application is `monthly_expense_imports`. The database name is configured using the `MONGO_DB` environment variable, with `monthly_expense_imports` as the default value.

The application stores import records in the `bank_statement_imports` collection.

Each import batch is represented as a document containing the following fields:

Field	Data Type	Description
`_id`	ObjectId	Unique identifier generated by MongoDB.

`userId`	Number	Identifies the user who owns the import batch.
`sourceName`	String	Name provided for the imported statement.
`importedAt`	Date	Date and time when the batch was submitted.
`status`	String	Represents the import-processing status.
`rowCount`	Number	Number of rows included in the batch.
`rows`	Array of objects	Contains the submitted statement rows.

The application defines the statuses `PENDING_REVIEW`, `CONVERTING`, and `CONVERTED`. A new batch created through the import API initially receives the status `PENDING_REVIEW`.

8.3 MongoDB Connection

The connection logic is implemented in `lib/mongo.ts`. The MongoDB Node.js driver is used to establish the connection.

The connection URI is obtained from the `MONGO_URI` environment variable. The database name is obtained from `MONGO_DB`, with a default value of `monthly_expense_imports`.

The exported function `getMongoDb()` provides access to the configured database. In development mode, the connection promise is

cached using `globalThis` to help avoid repeated connections during development reloads.

Database configuration is maintained in environment variables instead of hard-coded connection settings.

8.4 Import API Implementation

The import API is implemented in `app/api/imports/route.ts`. It provides two main operations: creating an import batch and retrieving a user's recent import batches.

8.4.1 POST Operation

The POST handler creates a new import batch in the `bank_statement_imports` collection.

The process follows these steps:

1. Checks whether the user has an authenticated session.
2. Reads and validates the submitted JSON request body.
3. Verifies that the statement name is a non-empty string of at most 255 characters.
4. Requires between 1 and 500 statement rows.
5. Checks that each submitted row is an object rather than null or an array.
6. Associates the batch with the authenticated user's ID.
7. Records the import timestamp, row count, statement name, rows, and initial status.
8. Inserts the batch into MongoDB and returns the generated document identifier.

A successful request returns HTTP status `201`. Invalid input returns `400`, unauthenticated requests return `401`, and server-side failures return `500`.

8.4.2 GET Operation

The GET handler retrieves the import batches belonging to the authenticated user.

It filters documents using the `userId` from the user's session, sorts the results by `importedAt` in descending order, and limits the result to the 20 most recent batches.

The API response includes each batch's identifier, statement name, import timestamp, status, row count, and a preview of at most the first three rows.

This allows the application to display recent imports without returning every row in each stored document.

8.5 Data Flow and Integration with MySQL

The statement import workflow connects the web interface, the backend API, MongoDB, and MySQL.

1. The authenticated user submits statement data through the application.
2. The import API verifies authentication and validates the submitted data.
3. MongoDB stores the import batch and its associated rows.
4. The application retrieves the user's recent batches for review.
5. Imported data can then be processed for conversion into structured MySQL transaction records.

MongoDB preserves the submitted import batch, while MySQL maintains the structured transaction records used for financial operations and reporting.

8.6 Why MongoDB Is Appropriate

MongoDB is used for statement imports because its document model can represent batches containing flexible row structures. This avoids requiring every submitted statement row to follow exactly the same set of fields at the storage stage.

MySQL is retained for the core financial entities because their relationships, keys, and integrity constraints are important for reliable financial record management.

The combination demonstrates the use of two database paradigms according to their respective data requirements, rather than using a second database solely for technology demonstration.

8.7 Security and Validation

The import API incorporates several protections:

- **Authentication:** Both GET and POST require an authenticated session.
- **User data isolation:** The GET query returns batches belonging only to the signed-in user.
- **Input validation:** The API validates the request structure, statement name, row count, and row types.
- **Configuration security:** The MongoDB connection URI is obtained from an environment variable.
- **Error handling:** The API returns appropriate HTTP error responses when validation or server operations fail.

The validation confirms that each imported row is an object, but it does not establish that every row contains valid financial data. Additional field-level validation is necessary before imported rows are converted into MySQL transactions.

8.8 Indexing Considerations

MongoDB automatically provides an index on the `_id` field. The displayed import API does not create any additional MongoDB indexes.

Because the import retrieval query filters by `userId` and sorts by `importedAt`, a compound index on these fields could be considered if the dataset grows:

```
{ userId: 1, importedAt: -1 }
```

This is a potential optimization and is not claimed to be an index already implemented in the project.

8.9 Conclusion

The MongoDB integration provides a flexible document store for bank statement import batches. Through the Next.js API, the application supports authenticated imports, input validation, user-specific retrieval, and import previews.

Together with MySQL, MongoDB allows RupeeWise to manage structured financial records and flexible statement-import data within the same application.

9. APPLICATION IMPLEMENTATION AND USER INTERFACE

9.1 Overview

RupeeWise is implemented as a web application using Next.js, React, and TypeScript. The application provides an interface for interacting with the financial records stored in MySQL and the statement-import documents stored in MongoDB.

The application separates the user interface, backend API routes, authentication logic, and database connection modules. This organization allows the interface to communicate with the databases through backend operations rather than connecting directly from the browser.

9.2 User Authentication

The authentication module supports user registration, login, session verification, and logout.

Passwords are hashed before storage using the `bcryptjs` library. After successful authentication, the application uses a signed session token stored in an HTTP-only cookie. Protected API routes verify the session before allowing access to restricted operations.

Authentication is used to associate financial records and statement-import batches with the signed-in user. When a user is not authenticated, protected routes return an unauthorized response.

9.3 Transaction Management

The transaction functionality supports the management of income and expense records.

Transactions are stored in the MySQL `transactions` table and associated with users and categories through foreign keys. Each record includes details such as transaction type, description, amount, transaction date, and optional notes.

Backend API routes support transaction operations and enforce user-specific access. Input validation and database constraints help prevent invalid records, while the database triggers verify that a transaction's type matches the type of its selected category.

9.4 Financial Dashboard and Budgeting

The financial dashboard provides an interface for accessing relevant financial information. The database supports monthly financial summaries through the `vw_monthly_financial_summary` view and the `sp_monthly_summary` stored procedure.

Monthly budget information is maintained in the `monthly_budgets` table. The application includes budget functionality for retrieving and updating the monthly budget associated with the authenticated user.

These components connect the application's financial management features with the underlying relational database.

9.5 Statement Import Interface

RupeeWise includes a statement-import interface that accepts JSON transaction rows and submits them for processing.

The import workflow uses the `/api/imports` API route. The backend validates the request, checks authentication, and saves valid batches in MongoDB's `bank_statement_imports` collection.

Users can retrieve their recent import batches, including their names, statuses, timestamps, row counts, and limited row previews. The application also supports converting validated imported rows into MySQL transaction records.

The current import interface accepts JSON rows; it should not be described as a direct CSV or bank-file upload feature unless that capability is implemented separately.

9.6 Backend API Integration

The application uses Next.js route handlers to process requests and connect the interface to the databases.

The backend is responsible for:

- Validating request data before processing it.
- Checking authenticated sessions for protected operations.
- Restricting data access according to the signed-in user's identity.
- Performing database operations through the appropriate connection modules.
- Returning structured JSON responses and appropriate error status codes.

MySQL is accessed through the application's relational database module, while MongoDB operations use the `getMongoDb()`function defined in `lib/mongo.ts`.

9.7 User Interface Screenshots

Actual application screenshots will be added after the final application walkthrough. The following screenshots should be included in this section or in the testing section of the report.

Figure 2: Login Page

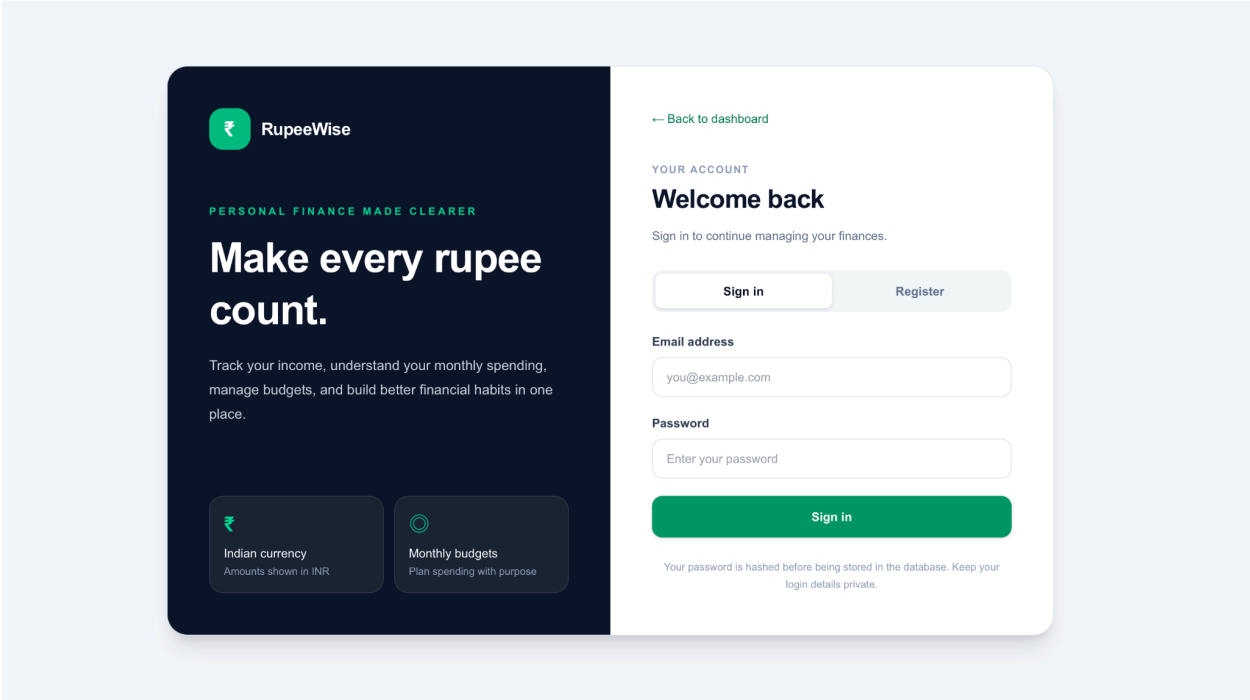


Figure 3: Financial Dashboard

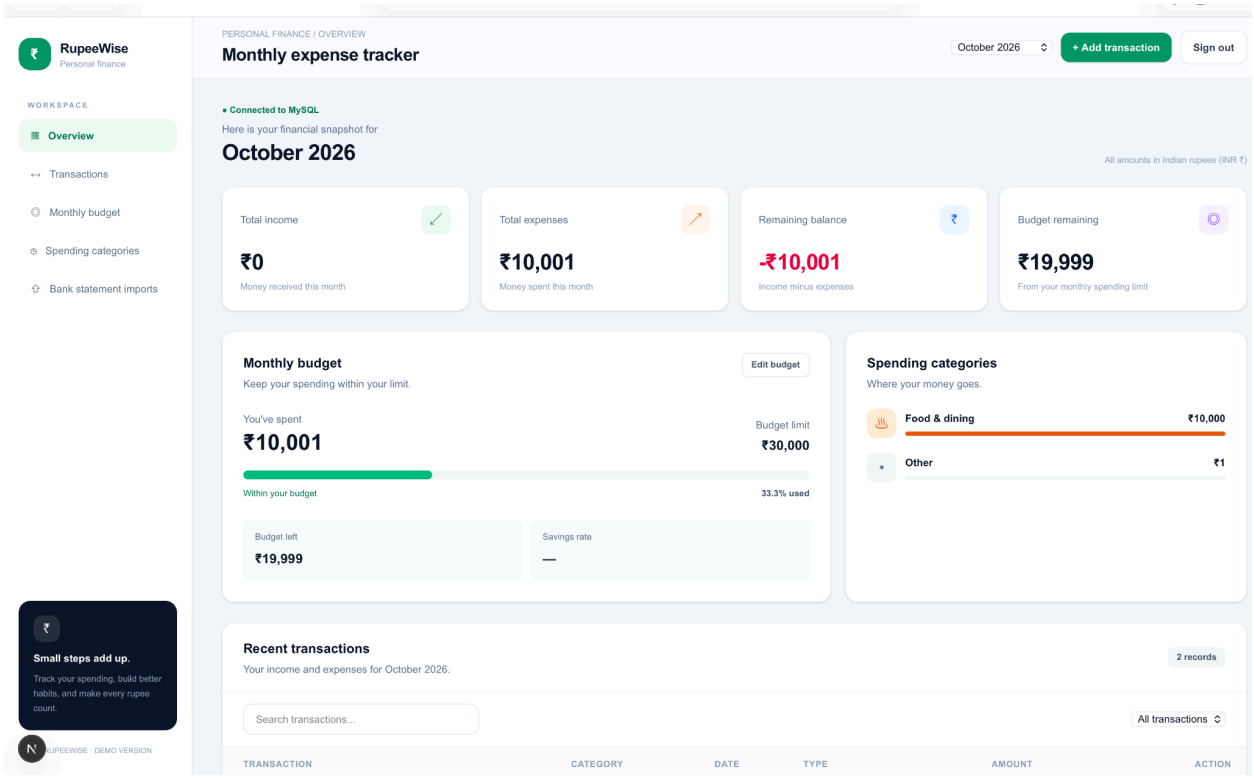
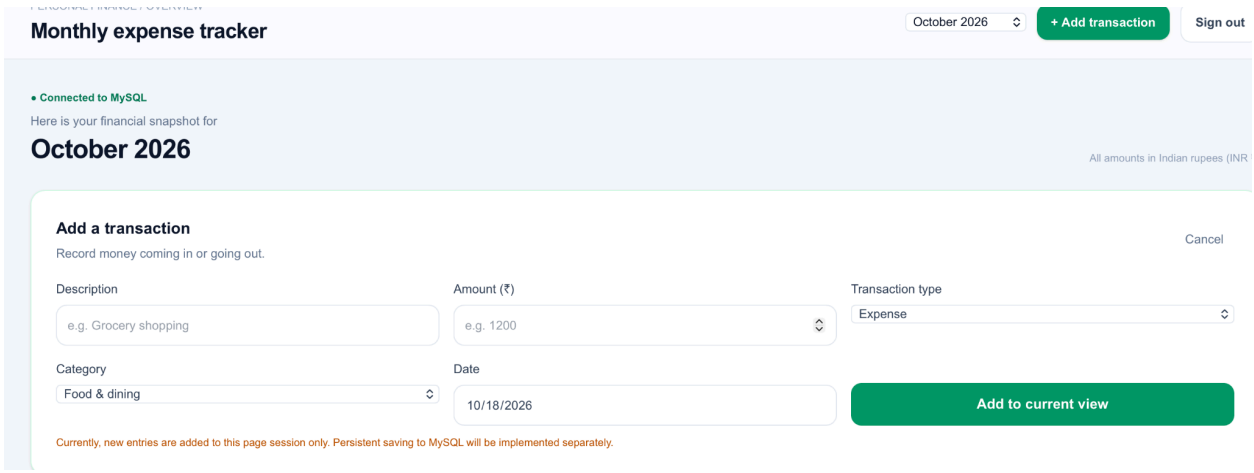


Figure 4: Transaction Management



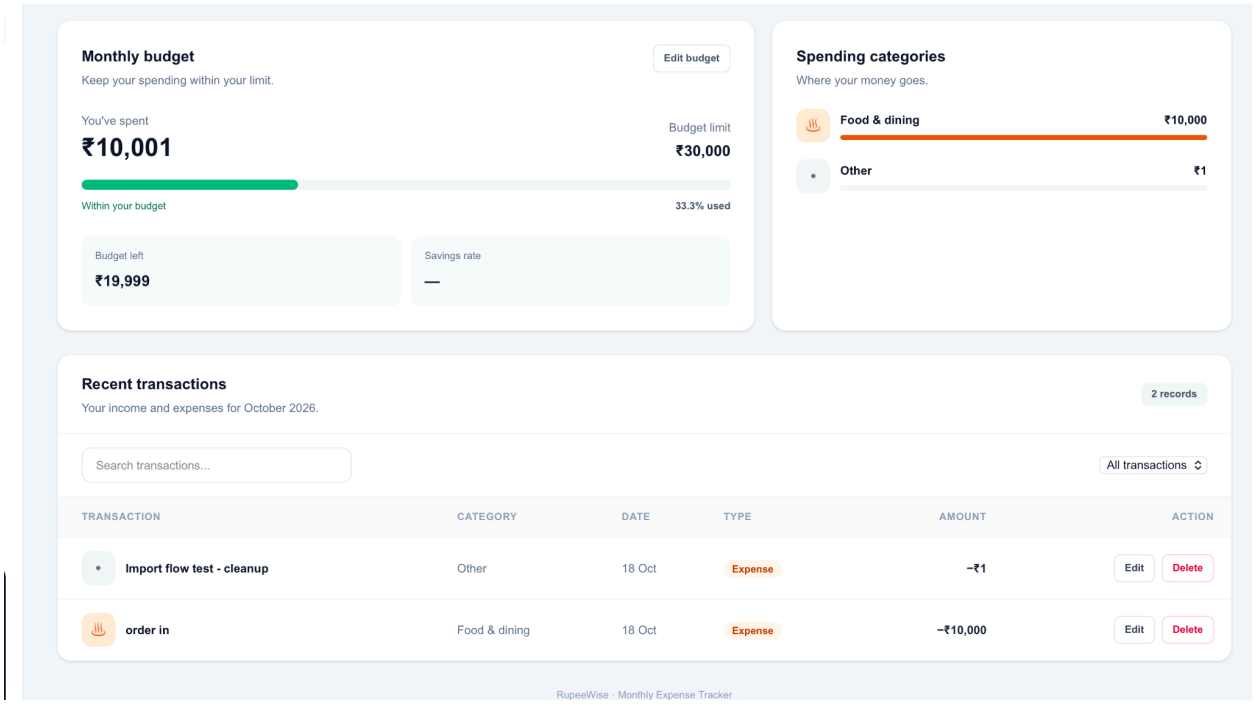


Figure 5: Monthly Budget Interface

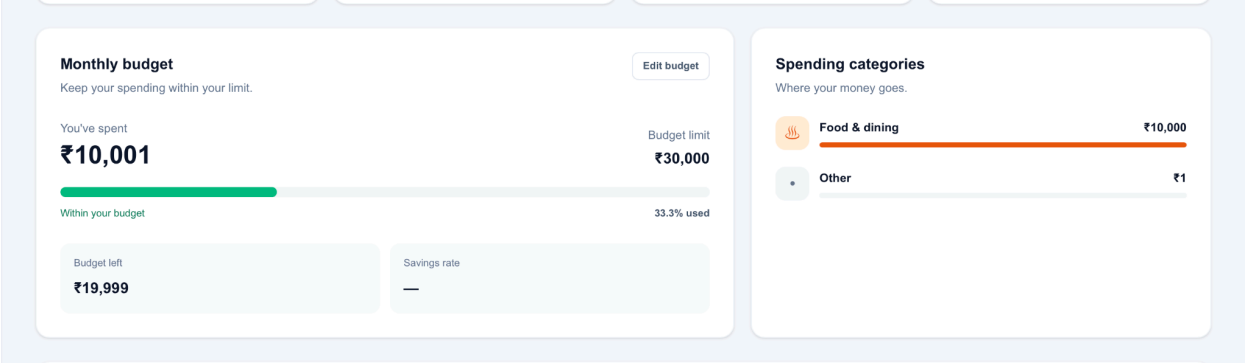
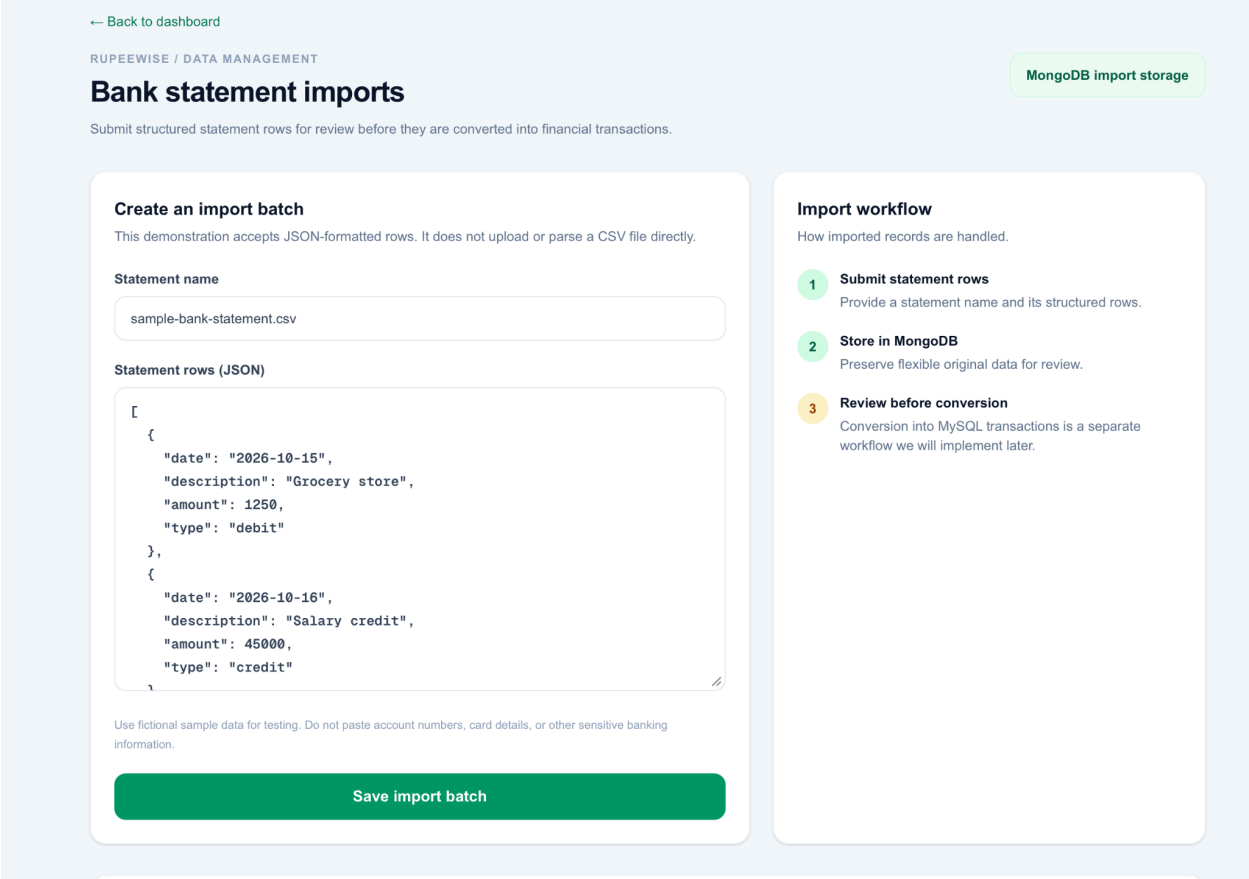


Figure 6: Statement Import Interface



9.8 Summary

The application implementation connects user-facing financial management features with backend API functionality and two database systems. Authentication, transaction management, budgeting, and statement imports demonstrate how database operations are integrated into a functional web application. The combination of structured MySQL records, MongoDB import documents, validation, and protected API routes provides the foundation for RupeeWise.

10. TESTING AND RESULTS

10.1 Introduction

Testing was carried out to check the functionality of RupeeWise, its database integration, authentication mechanisms, data validation, and application build. The tests focus on verifying that the application processes valid requests, rejects unauthorized access, maintains user-specific records, and uses the MySQL and MongoDB databases as intended.

Testing covered application authentication, transaction operations, monthly budget functionality, statement imports, advanced SQL features, and production build verification.

10.2 Testing Environment

The project was developed and tested using the following environment:

Component	Technology
Operating System	macOS
Runtime	Node.js
Application Framework	Next.js
Programming Language	TypeScript
Relational Database	MySQL 8.4.11

NoSQL Database	MongoDB
Database Interface	MySQL client and application API routes
Version Control	Git and GitHub

10.3 Functional Test Cases

The following table summarizes the functional areas tested during development.

Test ID	Test Case	Expected Result	Recorded Result
TC 01	User registration	A valid registration creates a user account.	Passed during development
TC 02	User login and session	Valid credentials establish an authenticated session.	Passed during development
TC 03	User logout	Logging out removes the authenticated session.	Passed during development
TC 04	Protected API access	Unauthenticated requests are rejected.	Passed during development
TC 05	Transaction CRUD	Authenticated users can create, retrieve, update,	Passed during development

		and delete their own transactions.	
TC 06	Monthly budget API	Authenticated users can retrieve and update their monthly budget.	Implemented; include a demonstration screenshot in the final submission
TC 07	MongoDB import creation	A valid import batch is saved in MongoDB.	Passed during development
TC 08	Import retrieval	An authenticated user retrieves their own recent import batches.	Implemented and tested during development
TC 09	Import conversion	Validated imported rows can be converted into MySQL transaction records.	Passed during development
TC 10	Database validation	Invalid transaction category/type combinations are rejected.	Trigger rejection test passed
TC 11	TypeScript validation	The application has no TypeScript errors.	Passed
TC 12	Production build	The application builds successfully for production.	Passed

The recorded results summarize the checks performed during development. Screenshots showing the corresponding outputs will be added to the final submission where applicable.

10.4 Database Testing

Database testing was performed to verify the relational schema, advanced SQL objects, and transaction validation rules.

10.4.1 Table and Relationship Verification

The database contains six relational tables: `users`, `categories`, `transactions`, `monthly_budgets`, `category_budgets`, and `savings_goals`.

Primary keys, foreign keys, unique constraints, check constraints, and indexes were defined to maintain consistency between records and support data retrieval.

10.4.2 Monthly Financial Summary View

The view `vw_monthly_financial_summary` was tested to verify that it groups recorded transactions by user and month and calculates total income, total expenses, and transaction count.

10.4.3 Monthly Summary Stored Procedure

The stored procedure `sp_monthly_summary` was tested to generate a monthly report for a selected user. It calculates income, expenses, net balance, and transaction count for the specified month.

10.4.4 Transaction Validation Triggers

The triggers `trg_transaction_category_insert` and `trg_transaction_category_update` were tested to ensure that transaction types match their selected categories.

An inconsistent transaction was rejected by the database validation trigger, demonstrating that this rule is enforced at the database level.

10.5 MongoDB Integration Testing

The MongoDB integration was tested through the statement-import API.

The application successfully created import batches in the `bank_statement_imports` collection and supported retrieving the authenticated user's batches. The import workflow was also tested for conversion of validated imported rows into MySQL transactions.

The API includes validation for request structure, statement name, row count, and row types. Authentication checks restrict import operations to signed-in users, and the retrieval query filters batches using the authenticated user's ID.

10.6 Build and TypeScript Verification

The following commands were executed during final technical verification:

```
npx tsc --noEmit
npm run build
```

Both checks completed without errors.

The TypeScript check verifies that the application passes the TypeScript compiler's checks. The production build verifies that Next.js can compile the application for production. These results

support the technical readiness of the codebase, although they do not replace end-to-end testing of every user interface feature.

10.7 Test Evidence and Screenshots

The final report will include genuine screenshots showing the results of selected tests.

Figure 7: Successful User Authentication

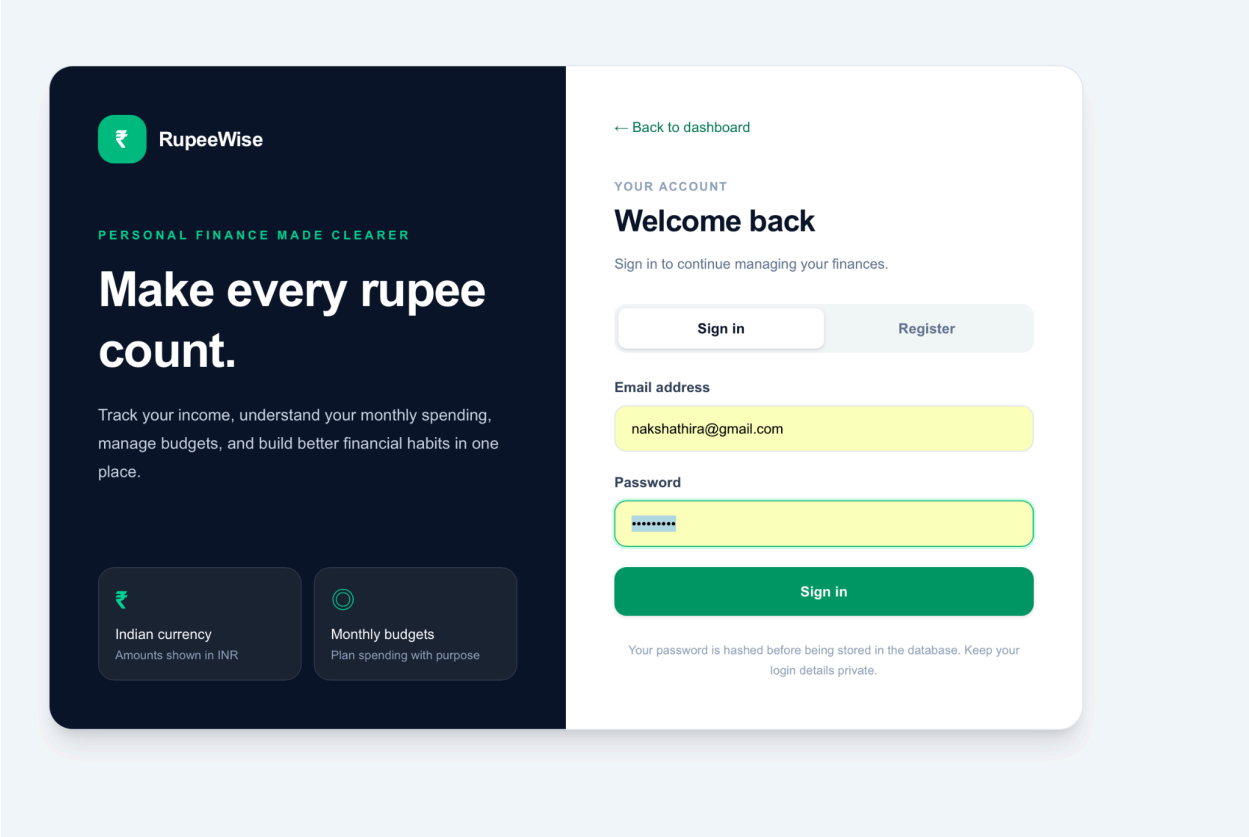


Figure 8: Transaction Management

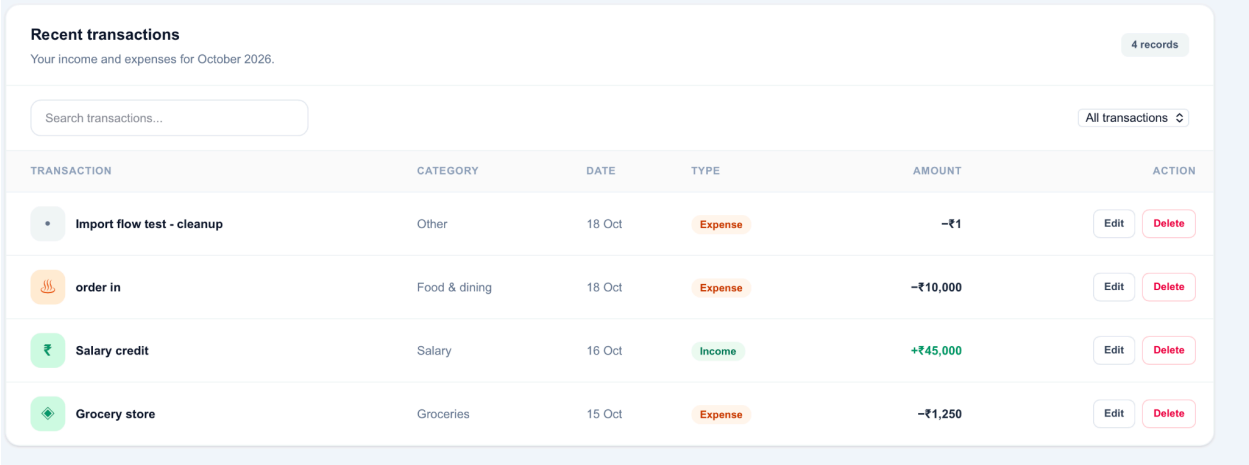


Figure 9: Monthly Financial Summary

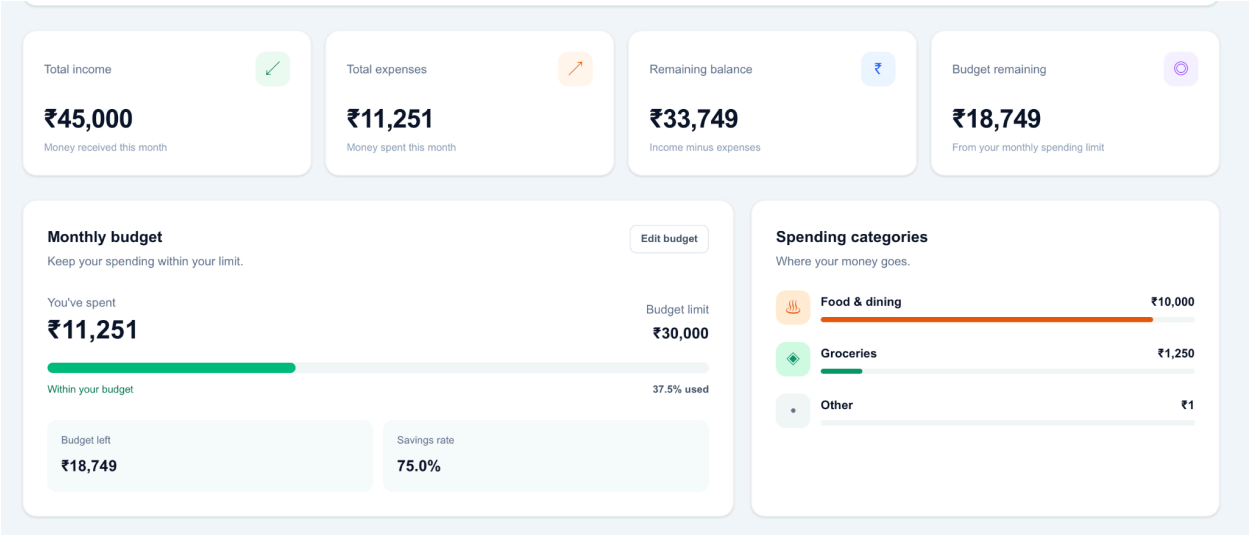
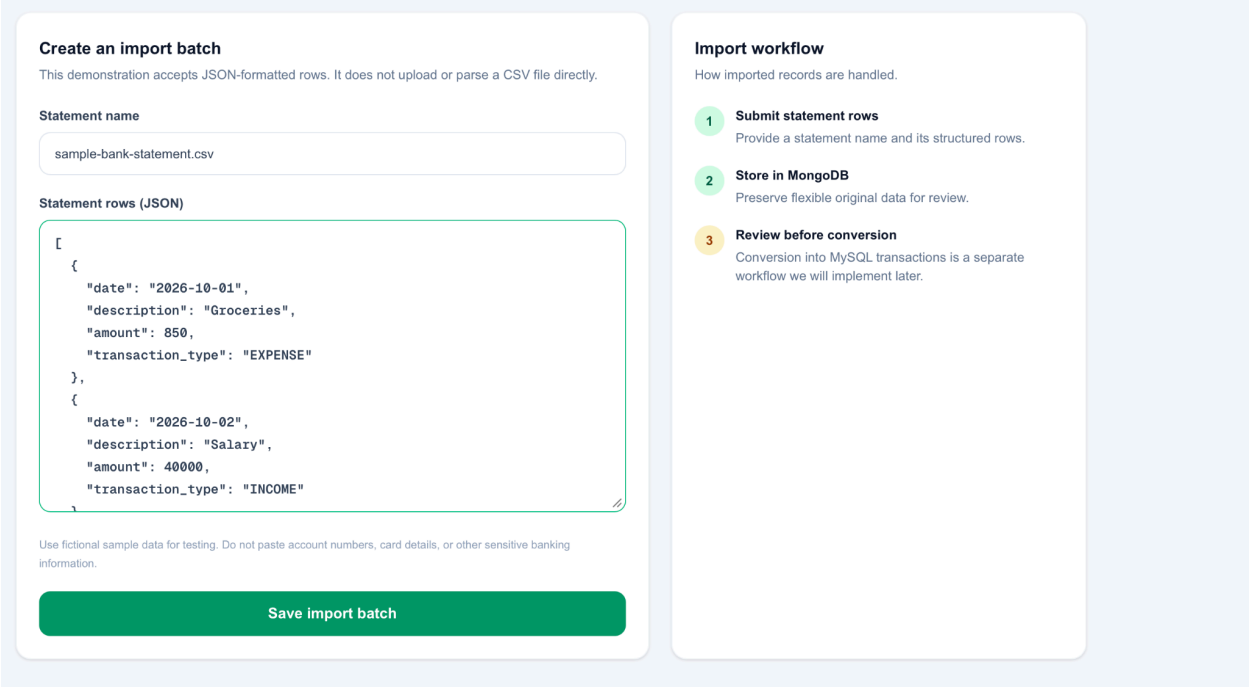


Figure 10: Trigger Validation

```
mysql> INSERT INTO transactions
    -> (user_id, category_id, transaction_type, description, amount, transaction_date)
    -> SELECT
    ->     u.user_id,
    ->     c.category_id,
    ->     IF(c.transaction_type = 'INCOME', 'EXPENSE', 'INCOME'),
    ->     'Trigger validation test',
    ->     1.00,
    ->     CURDATE()
    -> FROM users u
    -> CROSS JOIN categories c
    -> LIMIT 1;
ERROR 1644 (45000): Transaction type must match category type
mysql>
```

Figure 11: MongoDB Import



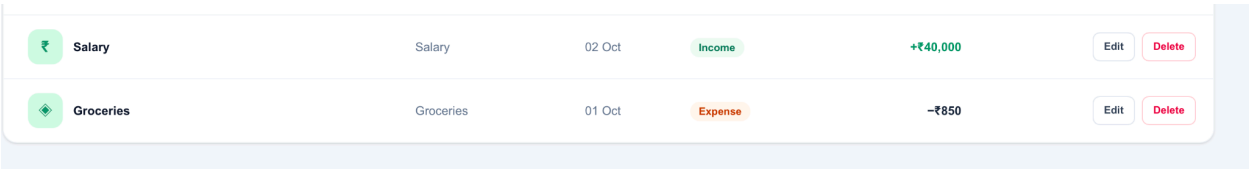
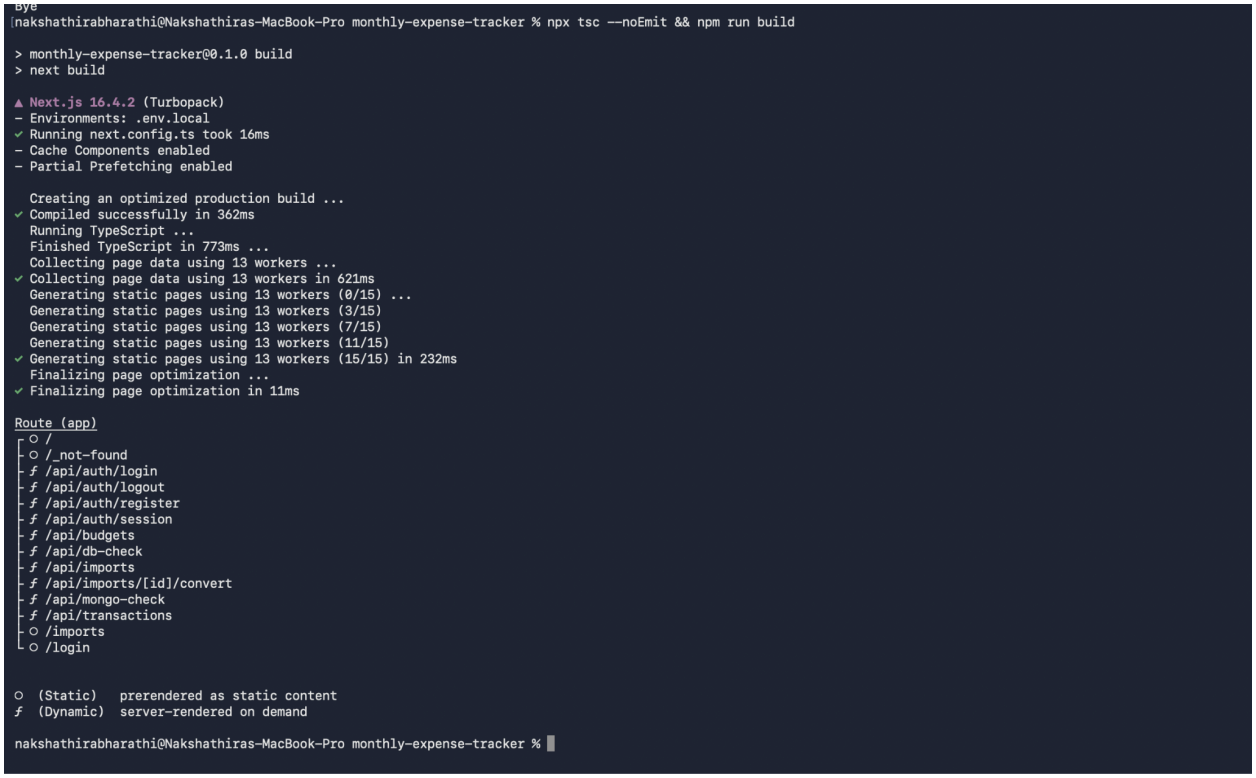


Figure 12: Production Build Verification



10.8 Results and Discussion

The tested application demonstrates authentication, protected transaction operations, monthly budget functionality, MongoDB statement imports, and conversion of imported rows into relational transaction records. The database also provides monthly reporting through a view and stored procedure, while triggers reject inconsistent transaction category/type combinations.

The successful TypeScript and production build checks indicate that the application compiles successfully. The MySQL and MongoDB integration demonstrates the use of relational and document-oriented storage within a common web application.

Further verification of individual user interface workflows and collection of final screenshot evidence will strengthen the submission and provide a clear demonstration of the system's behavior.

10.9 Conclusion

Testing confirms that key RupeeWise application and database functions were exercised during development. Authentication, transaction operations, statement import workflows, advanced SQL validation, and build verification were included in the testing process.

The results demonstrate the practical application of database design, integrity enforcement, API integration, and SQL and NoSQL technologies in a personal financial management system.

11. CONCLUSION AND FUTURE ENHANCEMENTS

11.1 Conclusion

RupeeWise — Monthly Expense Tracker was developed to address the practical problem of organizing and monitoring personal financial information through a web-based application. The project combines application development with relational and NoSQL database technologies to demonstrate the practical use of Database Management System concepts.

MySQL serves as the primary database for structured financial information, including users, categories, transactions, monthly budgets, category budgets, and savings goals. The relational schema incorporates primary keys, foreign keys, unique constraints, check constraints, and indexes to support data integrity and efficient data retrieval.

The project also demonstrates advanced SQL functionality through the `vw_monthly_financial_summary` view, the `sp_monthly_summary` stored procedure, and insert and update triggers that validate transaction category types. These features support reusable financial reporting and enforce important consistency rules at the database level.

MongoDB is integrated to store bank statement import batches in a document-oriented format. Through Next.js API routes, the application supports authenticated import operations, validation, user-specific retrieval, and processing of imported rows for conversion into structured MySQL transactions.

The application brings together authentication, transaction management, budgeting functionality, financial summaries, and statement imports. TypeScript validation and the production build were also completed successfully during development.

Overall, RupeeWise demonstrates how database design, SQL operations, advanced database concepts, NoSQL integration, API development, and security practices can be combined to address a real-world personal finance management problem.

11.2 Future Enhancements

Although RupeeWise provides a foundation for personal financial tracking, several improvements could extend its functionality.

1. Direct Bank Statement File Uploads

Support for CSV and other structured bank statement formats could reduce the need to enter statement rows as JSON. File parsing and additional validation could make the import workflow more convenient.

2. Advanced Spending Analytics

Additional reports and visualizations could help users identify spending patterns, compare monthly expenditure, and analyze category-wise trends over time.

3. Budget Alerts

The application could notify users when their expenditure approaches or exceeds a monthly or category-specific budget.

4. Improved Savings Goal Tracking

Additional progress indicators, contribution history, and goal-completion summaries could help users monitor their savings more effectively.

5. More Comprehensive Import Validation

The import workflow could include stricter field-level validation, duplicate transaction detection, and improved error reporting before records are converted into MySQL transactions.

6. Administrative Functionality

A dedicated administ

12. REFERENCES

1. **Next.js Documentation.** Next.js framework documentation. Available at: https://nextjs.org/docs
2. **React Documentation.** React library documentation. Available at: https://react.dev/
3. **TypeScript Documentation.** TypeScript language documentation. Available at: https://www.typescriptlang.org/docs/
4. **Oracle.** MySQL 8.4 Reference Manual. Available at: https://dev.mysql.com/doc/refman/8.4/en/
5. **MongoDB, Inc.** MongoDB Manual. Available at: https://www.mongodb.com/docs/manual/
6. **MongoDB, Inc.** MongoDB Node.js Driver Documentation. Available at: https://www.mongodb.com/docs/drivers/node/current/
7. **Node.js Documentation.** Node.js runtime documentation. Available at: https://nodejs.org/docs/
8. **Tailwind CSS Documentation.** Tailwind CSS styling framework documentation. Available at: https://tailwindcss.com/docs
9. **Git Documentation.** Git version control documentation. Available at: https://git-scm.com/doc
10. **DBMS Experiential Learning — Level 3 Project Requirements.** Course project requirements document provided for the Database Management Systems project.
11. **RupeeWise Source Code Repository.** Monthly Expense Tracker project. Available at: https://github.com/nakshathira200-svg/monthly-expense-tracker
12. **RupeeWise Project Documentation.** Data Dictionary, SQL Implementation, and MongoDB Implementation documents prepared for this project.